

操作系统

第3章 外设与文件

(1) 外设管理 (上)

同济大学计算机系

Part 1

编程外设硬件

▪ 计算机的外部设备 (External Device) — CPU 和 内存之外的所有硬件

▪ CPU 访问内存 和 访问外设 是不一样的

- 内存中的数据，每个字节有物理地址。以字节为单位随机访问，每个单元读写速度相等（UMA）。访问方式：CPU将物理地址放在地址总线上，直接访问。
- 外设中的数据没有地址，不能直接访问。需要使用特殊的方式 (in、out指令或内存映射)，先将外设中的数据取入内存或寄存器，之后再行访问。通常，外设中的字节不能随机访问。比如（1）键盘、网络输入信息是流式的，读完一个字节才能读下个字节（2）硬盘中的信息按扇区组织，扇区512字节，全读出来或者被新数据块覆盖。
- 访问外设中特别慢，慢数量级。∴ 设计高性能计算机系统必需考虑弥合内存/外设访问速度的差异。

1、计算机与外设的交互

外设连外设控制器，外设控制器连计算机



外设把物理世界的 声、光、电、磁、压力、温度、湿度.....一切信号转换成计算机系统能够处理的数字信号。

外设控制器把外设转换来的数字信号变成计算机系统能够处理的字节，存放在数据缓存里。

CPU向外设控制器发**IO**命令，外设控制器控制外设执行输入输出操作，完成后**CPU**读取数据缓存中的数据。

外设控制器的数据缓存和 IO 操作

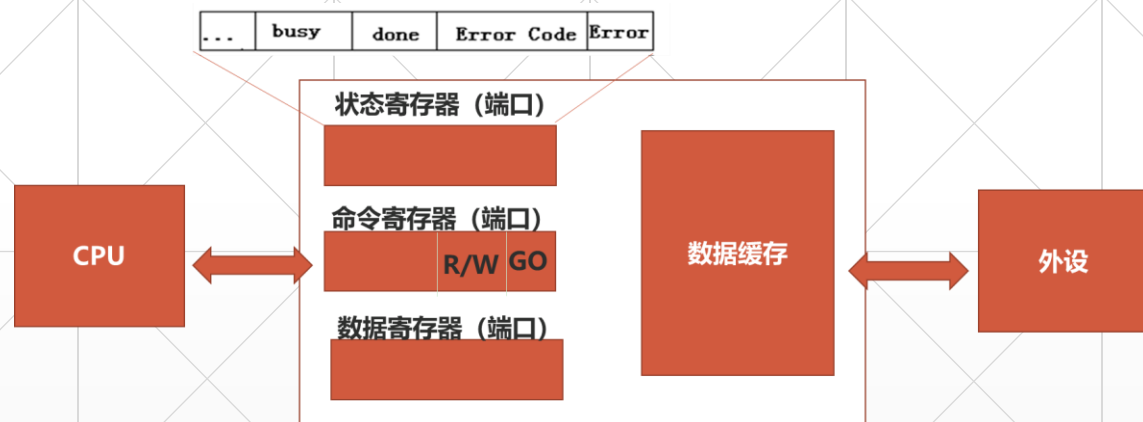


读操作 (I) : 外设转换好的数据放在数据缓存中, 供计算机读取

写操作 (O) : 计算机写数据缓存, 外设将其中的数据转化成物理信号向外界输出

外设一次执行一条IO命令, 完成或失败后, CPU才能发下条命令。

外设控制器的寄存器（端口），以磁盘为例



命令端口

- **R/W**: 1, 读命令; 0, 写命令
- **GO**: 开关

CPU写命令端口, 向外设发命令

状态端口

- **busy**: 1, 外设忙、正在执行IO操作; 0, 空闲
- **done**: IO完成
读操作, 数据缓存满, 是外设准备的新数据
写操作, 数据缓存空, 外设已将数据送至计算机外部
- **error** 标志为1, IO命令出错, **error code** 错误的类型

CPU访问状态端口, 查询、设置外设工作状态

数据端口（假设长度是1个整数，4字节）

- **CPU**读数据端口取数据缓存中的4字节（外设输入）
- **CPU**写数据端口向数据缓存写4字节（外设输出）

2、三种外设控制方式

- 程序直接控制 IO
 - 忙等 (busy waiting)
 - 轮询 (polling)
- 中断驱动的IO
 - 同步读
 - 异步写
- DMA控制的IO

2.1 忙等，读操作

CPU侧 (内核)

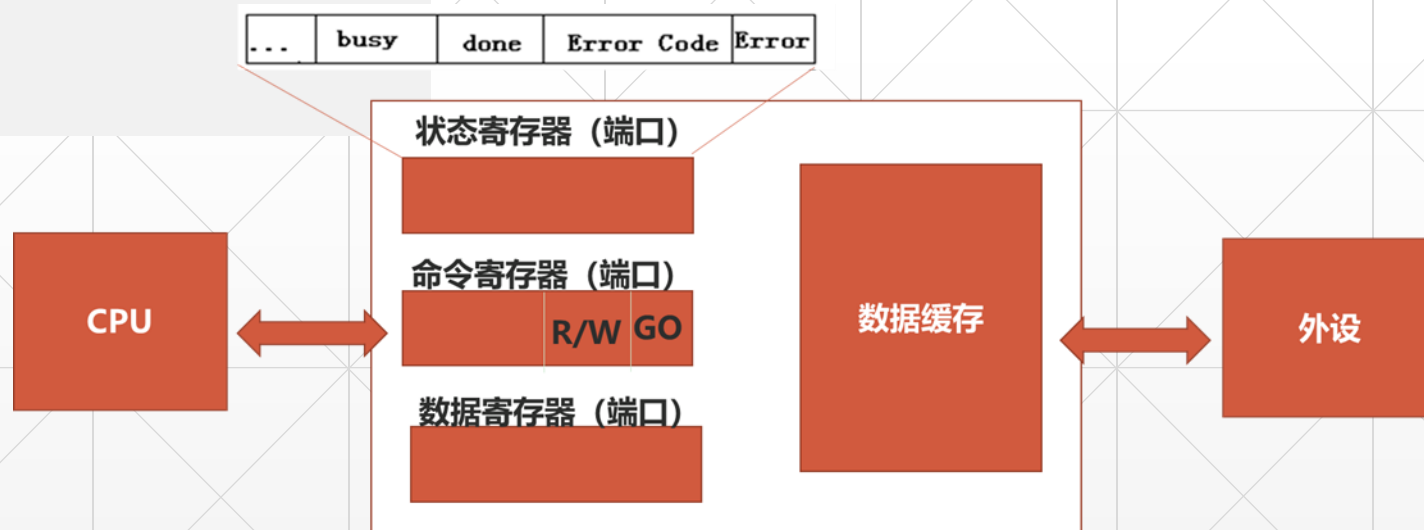
- 1、访问状态寄存器，忙等外设空闲 (busy 为0)
置外设忙: busy置1, done、Error和Error Code清0
- 2、写命令端口，发读命令
- 3、忙等 IO 完成 (done变1)
- 5、读数据缓存的内容到内存
- 6、busy标志清0，标记外设空闲

外设控制器

- 3、受令。控制外设，实施读操作
- 4、IO完成，done置1

外设硬件 (3)

- 信号转换
- 写数据进数据缓存



外设编程，一次磁盘读操作

1、忙等外设空闲

```
mov 0x0...0001000, flags
do {
    in statusPort, %eax
    and flags, %eax
} while (%eax)
```

2、置外设忙

```
mov 0x0...0001000, %eax
out %eax, statusPort
```

3.1 发读命令

```
mov 0x0...0000011, %eax
out %eax, commandPort
```

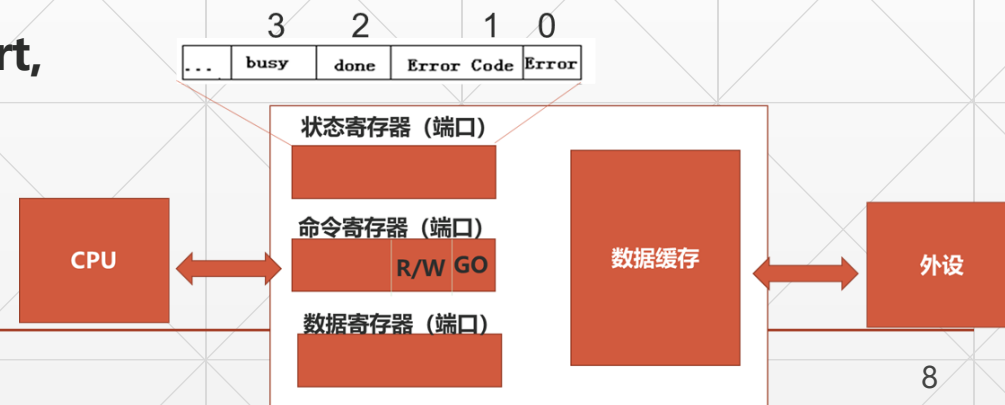
3.2 发写命令

```
mov 0x0...0000001, %eax
out %eax, commandPort,
```

4、读数据缓存的内容到内存，硬盘为例

数据缓存：512字节。 数据端口：4字节

```
数据端口读128次:
in dataPort, %eax
mov %eax, buffer[i]
inc i
```



2.2 忙等，写操作

CPU侧 (内核)

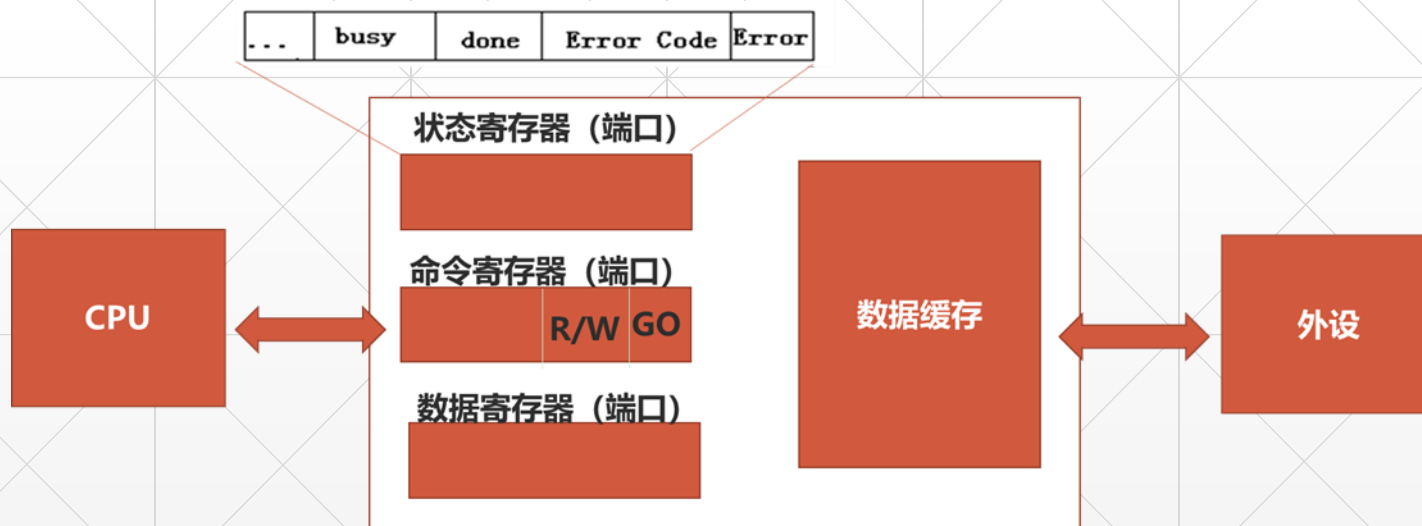
- 忙等外设空闲 (busy为0)
- 置外设忙: busy标志赋1, 此外done, Error 和 Error Code 清0
- 把内存数据写入数据缓存
- 发写命令
- 忙等 IO 完成 (done变1)
- busy标志清0

外设控制器

控制外设，实施写操作
IO完成，外设控制器将done标志置1

外设硬件

- 读数据缓存中的数据
- 信号转换送计算机外部



评论： 程序直接控制IO方式 忙等



2、 三种外设控制方式

2.1 程序直接控制IO方式 (忙等, 读操作)

- 忙等外设空闲 (busy为0)
- busy标志赋1, done标志清0
- 发读命令
- 忙等 IO 完成 (done变1)
- 读数据缓存的内容到内存
- busy标志清0



程序直接控制IO方式 (忙等, 写操作)

- 忙等外设空闲 (busy为0)
- busy标志赋1, done标志清0
- 把内存数据送入数据缓存
- 发写命令
- 忙等 IO 完成 (done变1)
- busy标志清0



CPU承担全部IO事务：查询外设状态、发IO命令，等待IO操作完成。
IO操作期间，CPU全程监控，无法执行其它任务：
计算、IO无法并行。多个外设无法并行工作。系统性能低下。

程序直接控制IO方式 2 轮询 (polling)

CPU承担全部IO事务：查询外设状态、发IO命令，等待IO操作完成。
IO时，CPU无需全程监控，可以执行与本次IO操作无关的其它任务：
计算、IO可以并行。多个外设也可以并行工作。系统高性能。

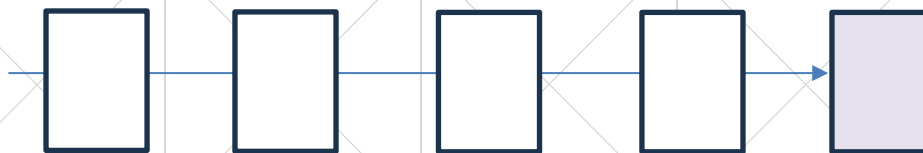
- 程序在合适的时间点主动查看外设状态
 - 发出请求时，设备忙，离开
 - 发出请求后，IO未完成，离开

2.3 中断驱动的IO方式（同步读）

将扇区blkno的内容读入内存，首地址为buffer，长度为512字节的内存空间

IO请求管理

FIFS的硬盘IO
请求队列



进程启动IO操作（前台，设备驱动）

- 1、现运行进程PA构造读IO请求块buf（mode: READ）送IO请求队列尾部。原先队列空？
 - Y，立即使用buf中的info，向磁盘发命令，启动IO
 - N，buf送队尾，等待队列前部IO操作完成
- 2、Sleep(&buf,-50)，入睡
- 3、处理底层送来的外设新数据

磁盘中断处理程序（后台，IO完成时唤醒等待IO操作完成的进程）

- 1、IO出错？
 - Y，重发队首buf的IO请求（试10次）
 - in数据端口读磁盘数据缓存，送buffer
- 2、WakeUpAll(&buf)，唤醒等待IO完成的进程
- 3、摘队首IO请求块
- 4、队列变空？
 - N，使用新队首buf中的info向外设发IO命令
 - Y，返回

Unix V6++的 IO请求块

```

class Buf
{
public:
    unsigned int b_flags; // B_READ
    int padding;
    Buf* b_forw; // 队列指针
    Buf* b_back;
    Buf* av_forw;
    Buf* av_back;

    short b_dev;
    int b_wcount; // IO数据量 (size, 512)
    unsigned char* b_addr; // 内存地址 (物理, buffer)
    int b_blkno; // 起始扇区号 (blkno)
    int b_error; // IO出错原因
    int b_resid; // I/O出错时尚未传送的剩余字节数
};
    
```

enum BufFlag

```

{
    B_WRITE = 0x1, // 写操作
    B_READ = 0x2, // 读操作
    B_DONE = 0x4, // I/O完成, 内存数据块与磁盘数据块一致, 数据可用
    B_ERROR = 0x8, // I/O出错
    B_BUSY = 0x10,
    B_WANTED = 0x20,
    B_ASYNC = 0x40,
    B_DELWRI = 0x80
};
    
```

/* b_flags中标志位 */

缓存块的使用状态

b_flags

...	B_ASYNC	B_BUSY	B_DONE	B_WRITE
	B_DELWRI	B_WANTED	B_ERROR	B_READ

同步读，IO命令发出后

CPU侧（设备驱动）

1、发IO命令

CPU侧（中断处理程序）

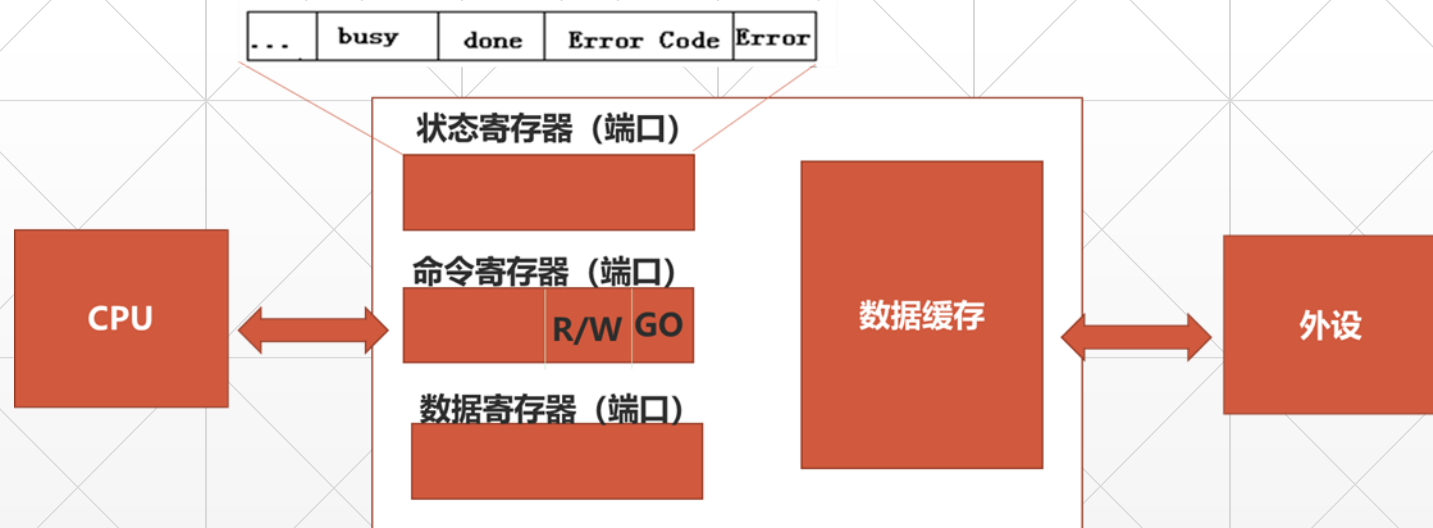
4、如前页PPT

外设控制器

- 2、控制外设，实施读操作
- 3、IO完成，外设控制器将done标志置1，发中断信号

外设硬件

2、准备好的数据送数据缓存



同步读操作的时序图



- t0时刻，copy进程执行**read系统调用上半段**：
时刻t1发磁盘读命令，入睡放弃CPU

[t1,t2], IO请求队列排队时间 + IO操作耗时

- t2时刻，IO完成。被中断的现运行进程PA执行**磁盘中断处理程序**唤醒copy进程。。。
- t3时刻，copy进程得到CPU**执行read系统调用下半段**，送数据进用户空间c变量。
- t4时刻，copy进程返回用户态执行应用程序。

或 执行进程图像换入、换出操作

2.4 中断驱动的IO方式 异步写

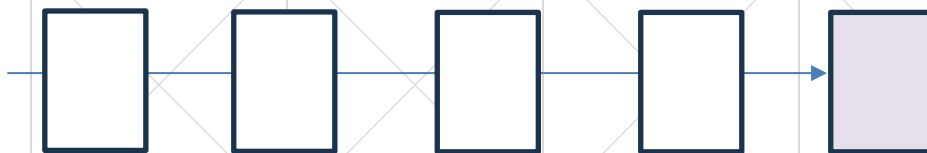
内存数据块写入磁盘

内存数据块：首地址为buffer，长度512字节

磁盘扇区：blkno

IO请求管理

FIFS的硬盘IO
请求队列



b_flags = B_WRITE | B_ASYNC

b_wcount = 512

b_addr = buffer

b_blkno = blkno

进程启动IO操作（前台，设备驱动）

1、现运行进程PA构造异步写IO请求块buf 送IO请求队列尾部。原先队列空？

Y，启动IO操作
N，buf送队尾

2、返回

（不睡，没有
同步等待）

启动IO操作：

1、取队首buf

- 读：向磁盘发读命令

- 写：out数据端口，将buffer中的内容写入磁盘数据缓存，向磁盘发写命令

磁盘中断处理程序（后台）

1、IO出错？Y，重发队首buf的IO请求（试10次）

2、摘队首IO请求块

读还是写？

- 读：in数据端口，将磁盘数据缓存中的内容送buffer 有异步标识？

- 没有，WakeUpAll(&buf)，唤醒等待IO完成的进程

3、队列变空了吗？

N，启动IO操作

异步写，总的流程（IO请求队列为空）

CPU侧

1、产生要写盘的数据

2、构造写请求块，送IO队列排队等待执行IO操作

3、将写盘数据送入数据缓存。

4、发写命令

CPU侧（中断处理程序）

6、none

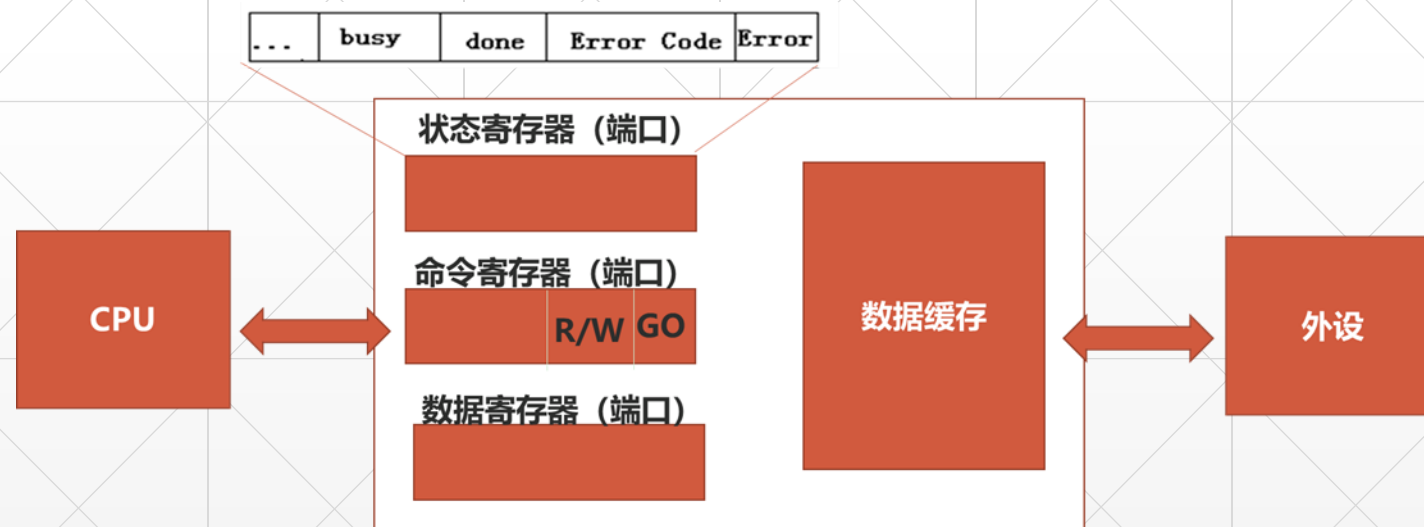
外设控制器

5、控制外设，实施写操作

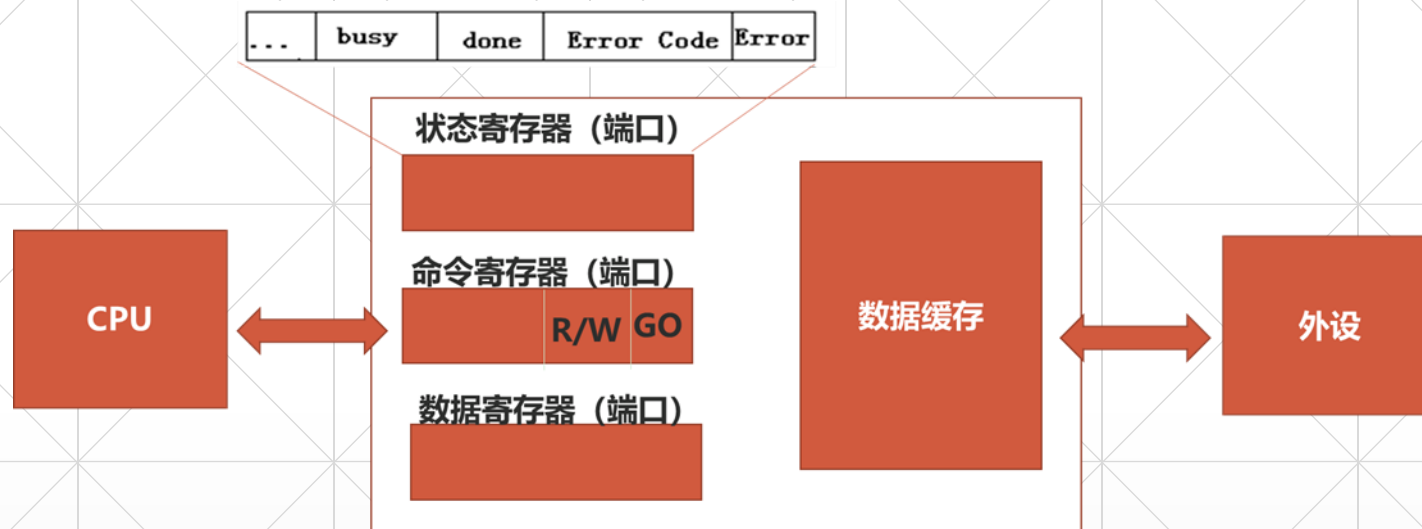
6、IO完成，外设控制器将done标志置1，发中断信号

外设硬件

5、读数据缓存中的数据
转换成磁信号写盘

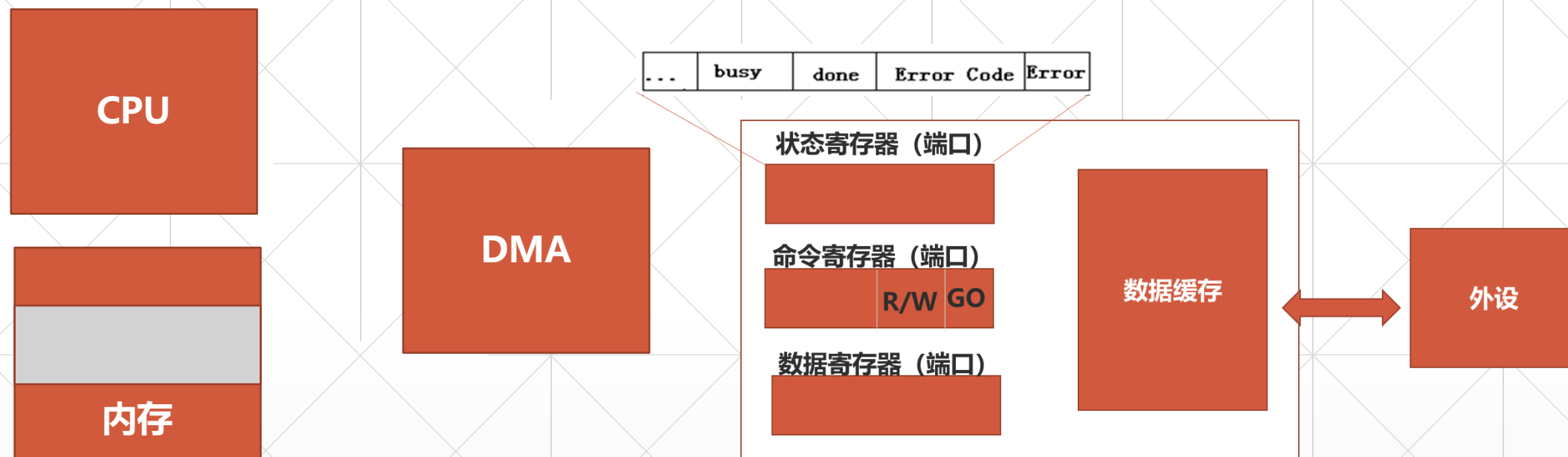


评论 中断驱动的IO方式



- 中断是IO完成信号。CPU无需忙等或轮询 IO操作。 计算和IO可以并行。不同的外设，IO也可以并行。
- CPU需要承担从外设数据缓存复制数据块到内存的工作。
- 中断驱动的IO方式适合键盘、鼠标 等数据缓存很小的外设。

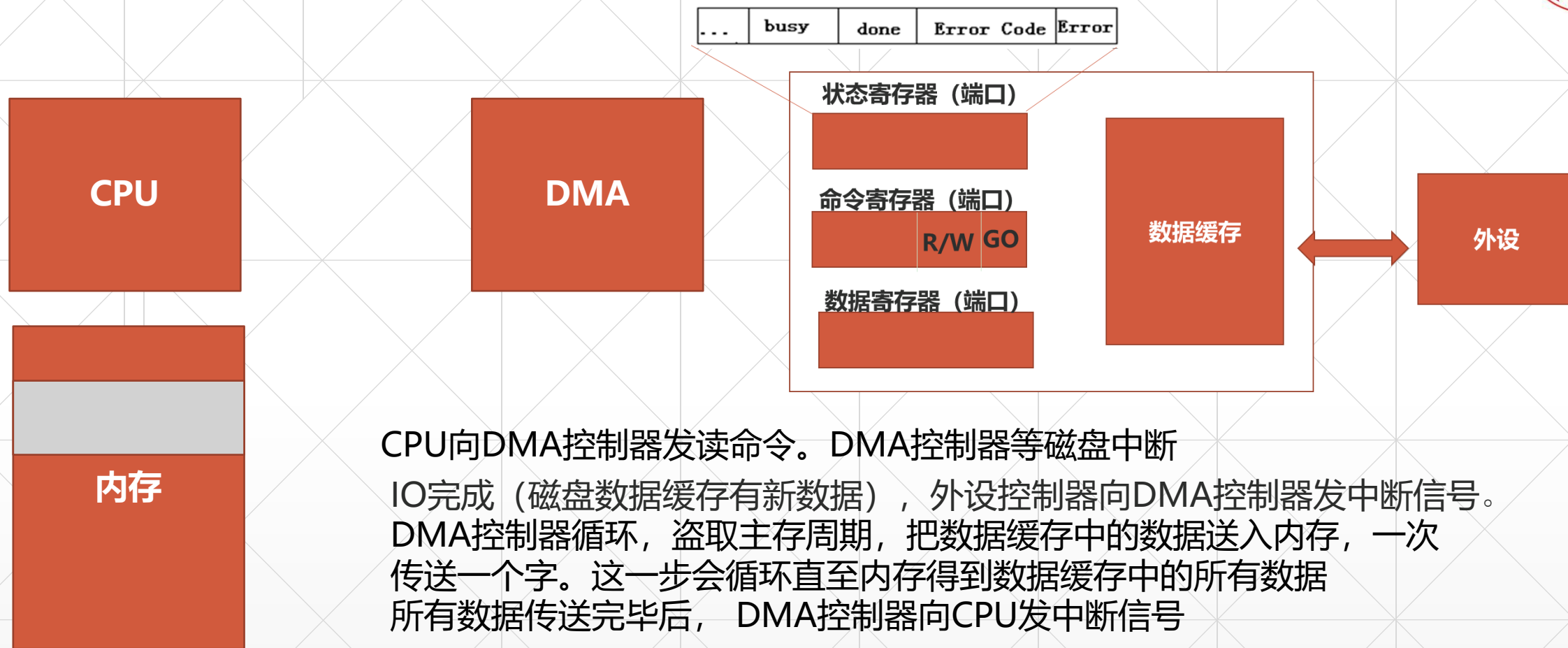
2.5 DMA (direct memory access)方式



DMA是数据复制专用芯片，专用于大批量数据复制场景，尤其适合：

- 磁盘数据缓存 $\leftrightarrow$ 内存
- 网卡数据缓存 $\leftrightarrow$ 内存
- GPU设备内存 $\leftrightarrow$ 内存（主机、其它GPU）

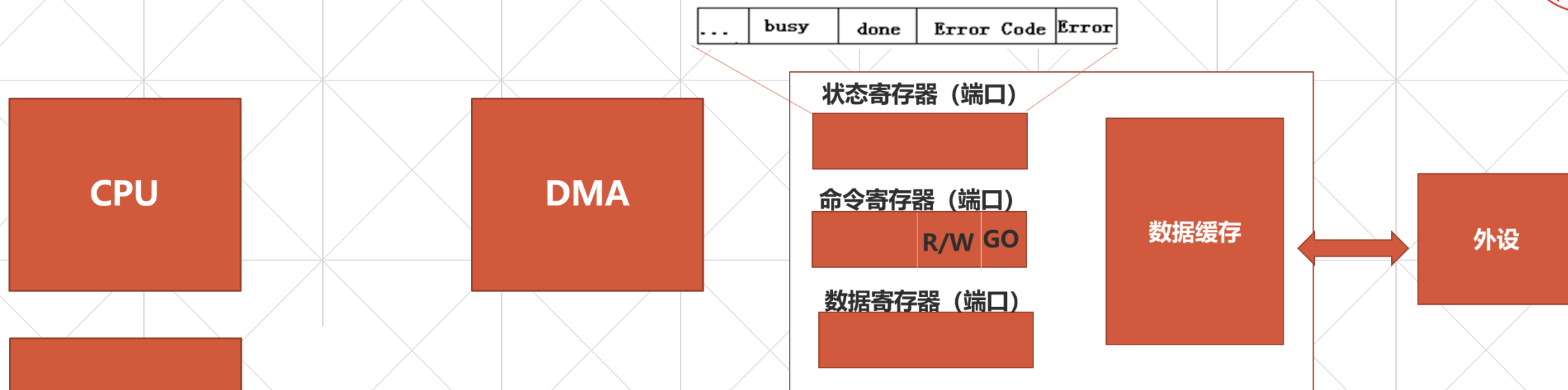
DMA读



CPU向DMA控制器发读命令。DMA控制器等磁盘中断IO完成（磁盘数据缓存有新数据），外设控制器向DMA控制器发中断信号。DMA控制器循环，盗取主存周期，把数据缓存中的数据送入内存，一次传送一个字。这一步会循环直至内存得到数据缓存中的所有数据所有数据传送完毕后，DMA控制器向CPU发中断信号

中断处理程序不复制数据。CPU完全不参与数据传输

DMA写

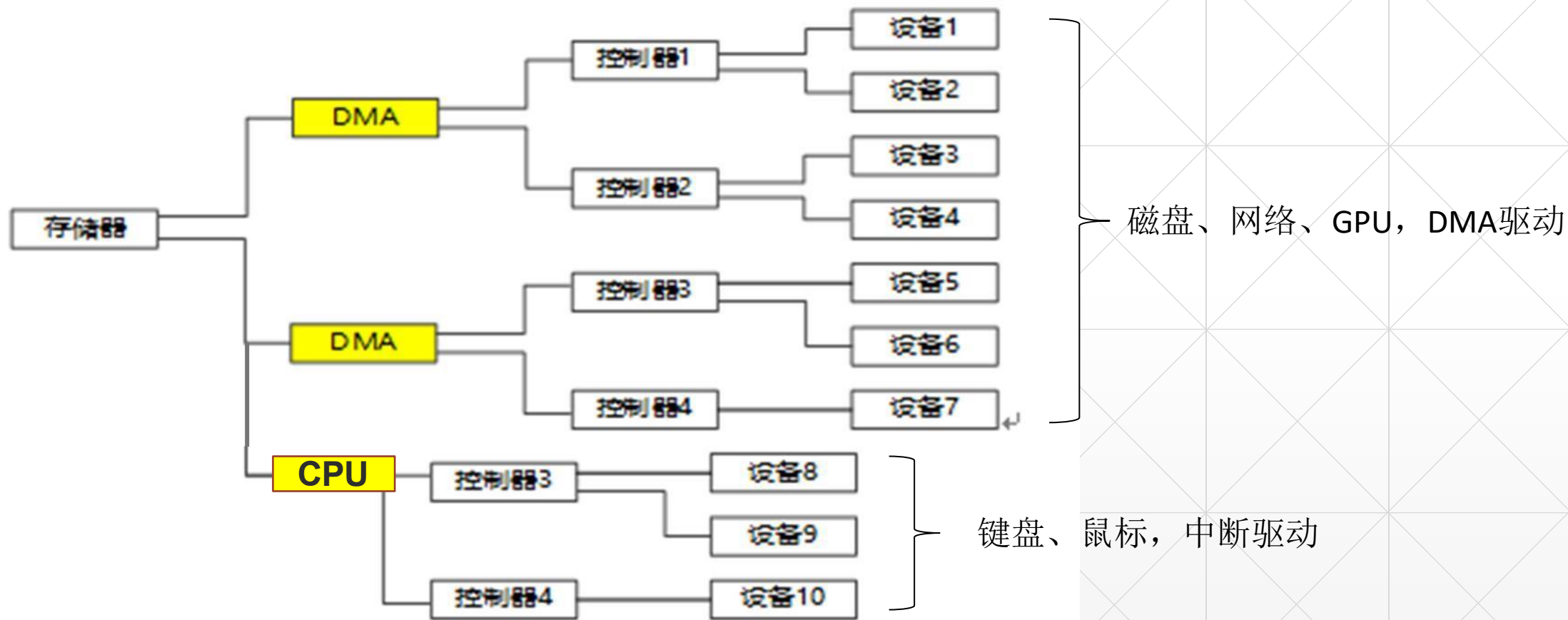


CPU向DMA控制器发写命令

- DMA控制器循环，盗取主存周期，把内存数据块写入磁盘数据缓存：一次传送一个字。这一步会循环直至磁盘数据缓存写满
- DMA控制器向磁盘控制器发写命令
- 磁盘控制器将数据缓存中的数据写盘，完成后，向DMA发中断信号
- DMA收到中断信号后。。。数据写完了？
 - 没有，再来一次。
 - 搞定，向CPU发磁盘中断，报告IO完成。

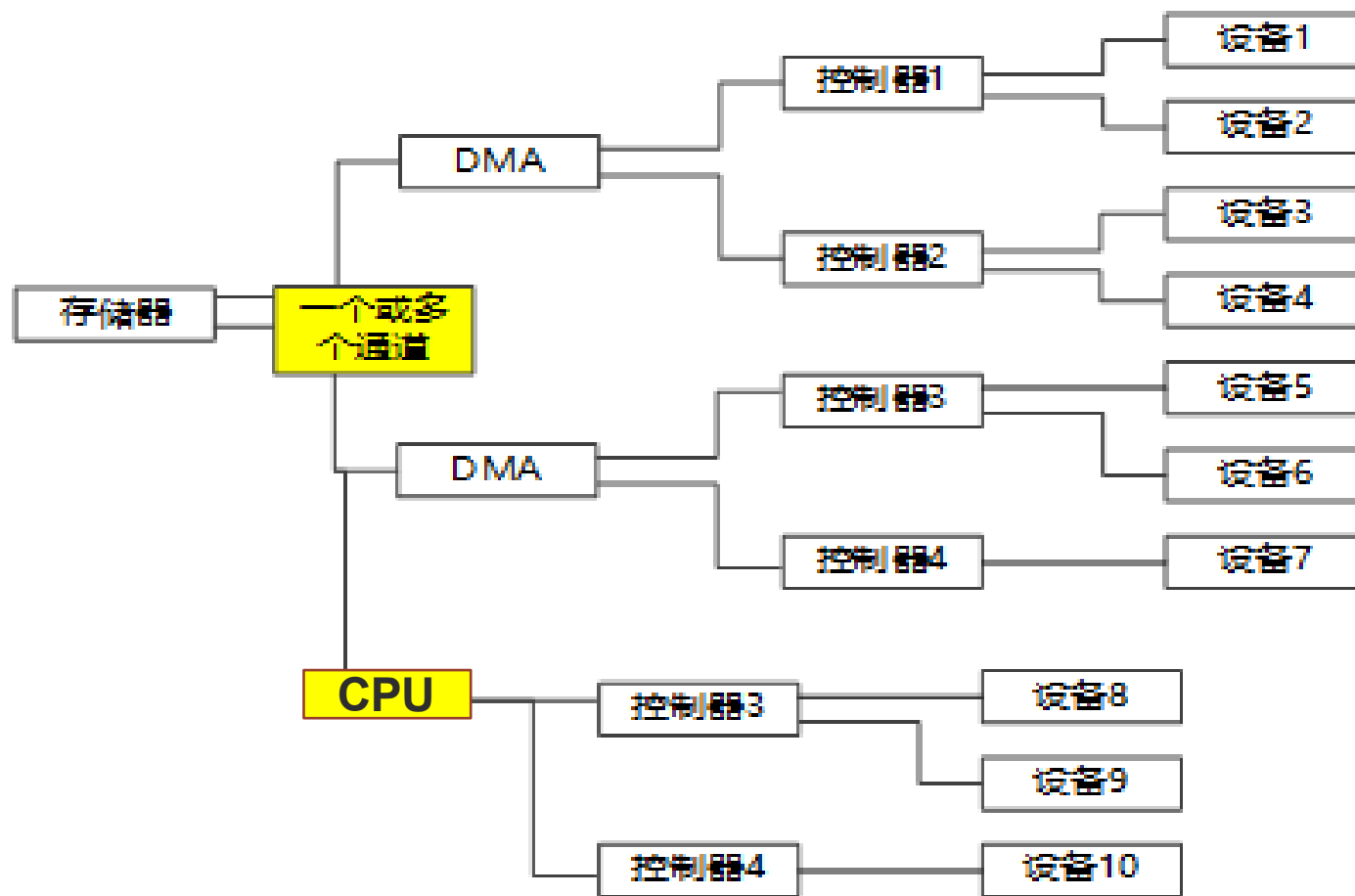
3 计算机外设系统硬件架构

PC 架构



计算机外设系统硬件架构

大型机/高档服务器架构





Part 2 缓存

- 内存中为每个外设配缓存，暂存外设IO数据。
- 缓存在核心空间。Unix V6++，在内核数据段。

1、不同外设，配备功能不同的缓存

- 块设备（存储设备）

- 用途 信息存储
- 举例 硬盘、软盘、U盘、光盘、磁带
- 特征 数据存储以数据块为单位
- 缓存(磁盘高速缓存池) 可重用的字符块集合

数据块是IO的最小单位。每次IO，读写的数据量是数据块尺寸的整数倍。

比如硬盘，Unix V6++的数据块是一个磁盘扇区，512字节；Linux的数据块是相邻的8个扇区，4096字节。

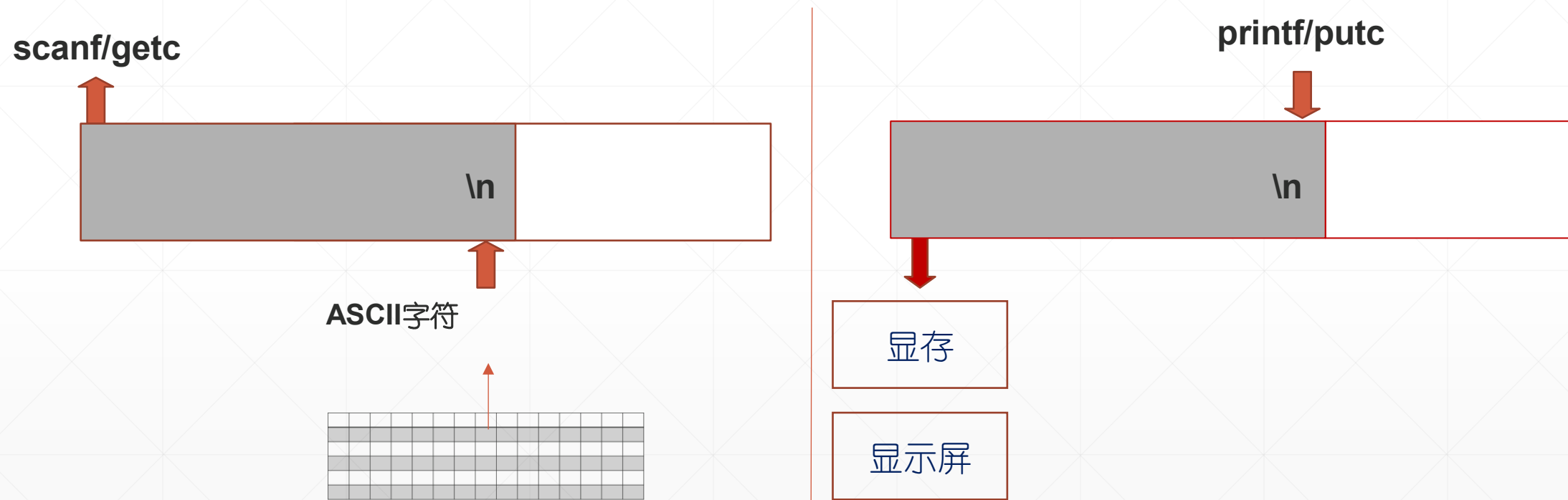
- 字符设备

- 用途 字符界面的人机接口
- 举例 终端设备(指键盘、屏幕)
- 特征 人机交互，每次一个字符行
- 缓存：字符缓存队列。

- 其它设备

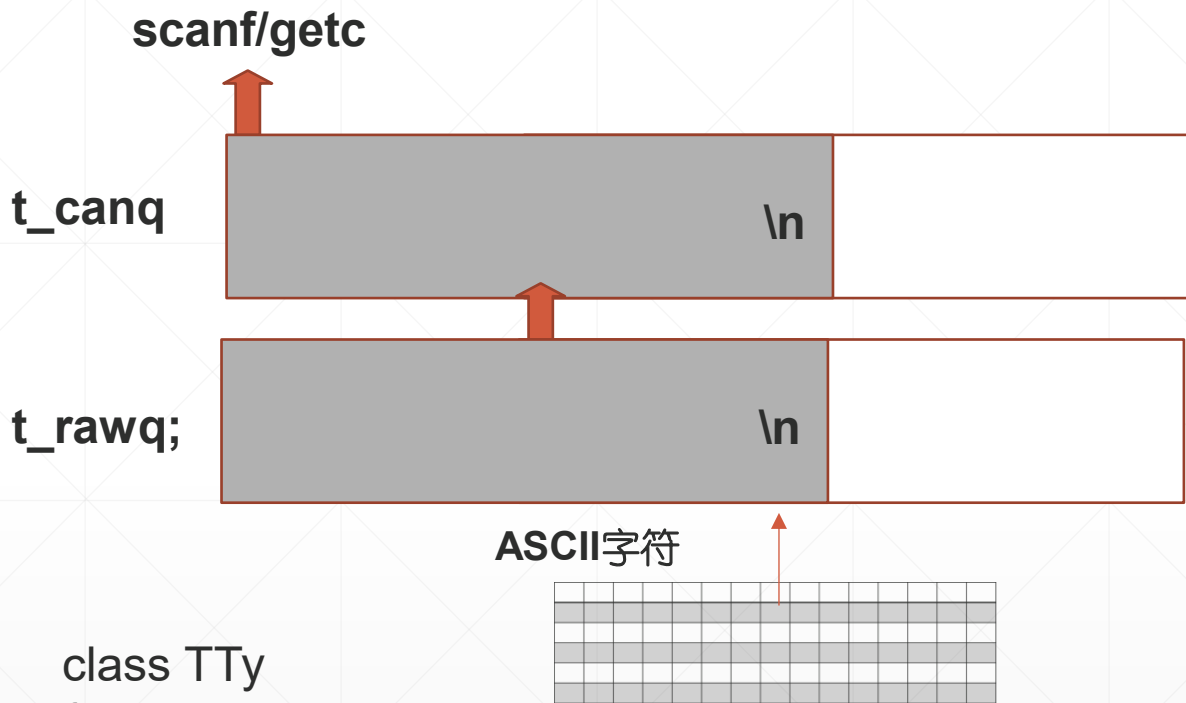
- 网络设备
- 鼠标
- 耳麦、扬声器、摄像头
- 其余传感器

2、字符设备的缓存 行缓存

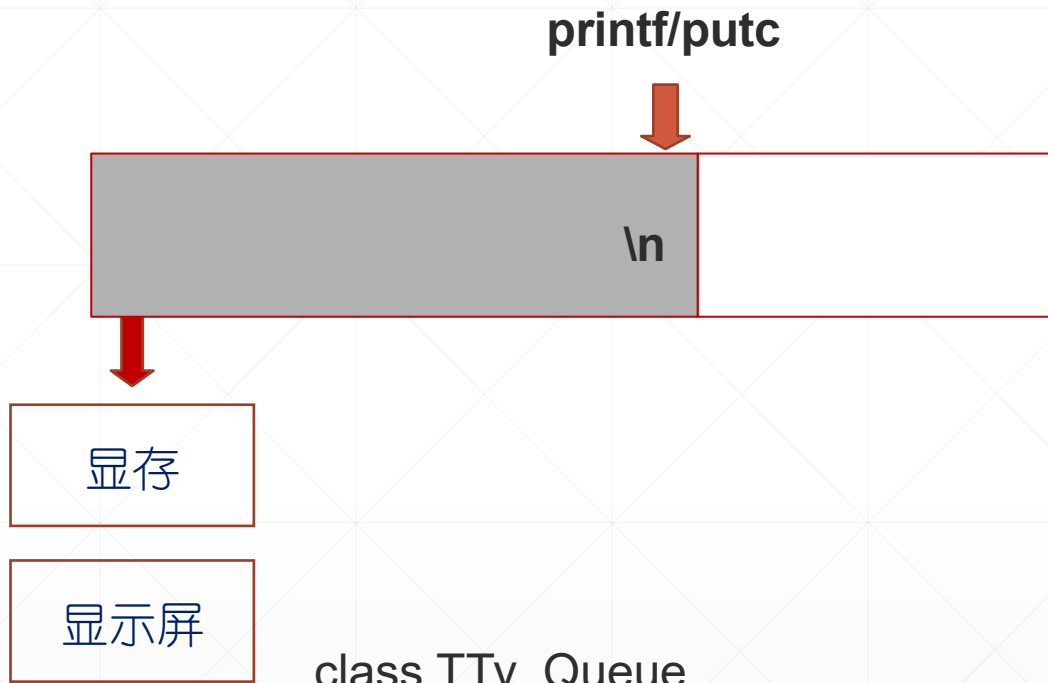


缓存的作用：减少进程切换次数，提高IO效率。

Unix V6++的行缓存



```
class TTy
{
    .....
    TTy_Queue t_rawq; /* 原始输入字符缓存队列 */
    TTy_Queue t_canq; /* 标准输入字符缓存队列 */
    TTy_Queue t_outq; /* 输出字符缓存队列 */
}
```

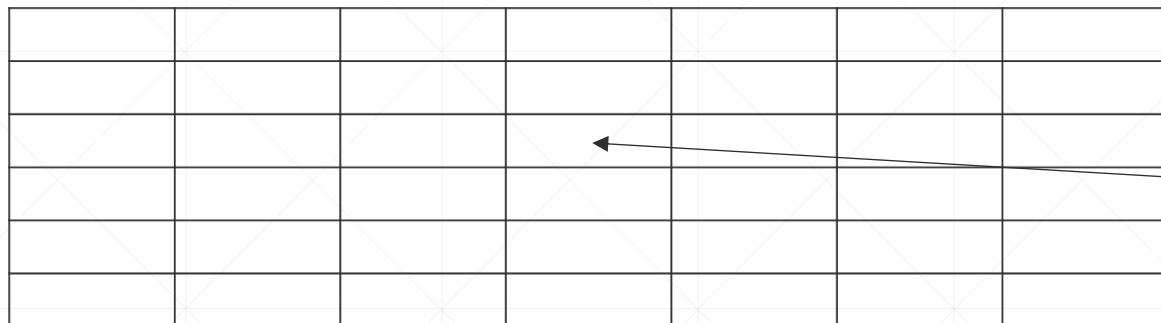


```
class TTy_Queue
{
    unsigned int m_Head;
    unsigned int m_Tail;
    char m_CharBuf[TTY_BUF_SIZE];
}
```

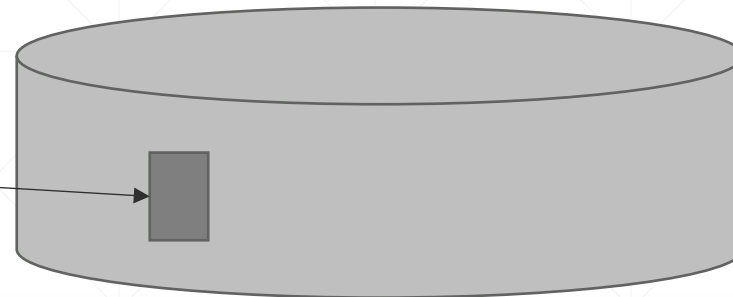
3、块设备的缓存

块缓存

内存设置**磁盘高速缓存池**（在内核空间），管理若干缓存块



磁盘

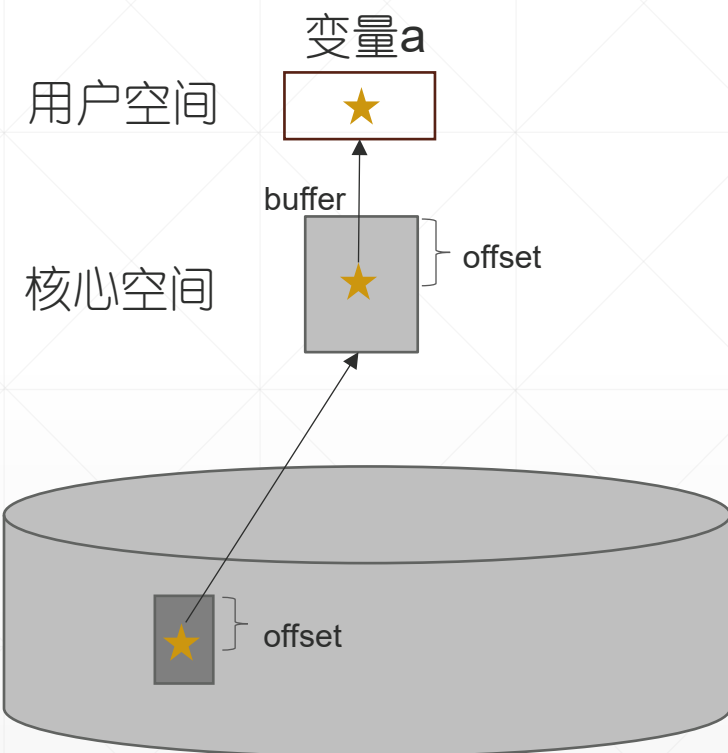


每个缓存块同步一个磁盘数据块，存放的数据复本，供所有进程使用。

一个磁盘数据块占据磁盘连续多个扇区。

- UNIX V6++, 缓存块512字节，同步磁盘上的1个扇区。
- Linux, 缓存块4096字节，同步连续8个扇区。

借助缓存的读操作（不考虑并发）

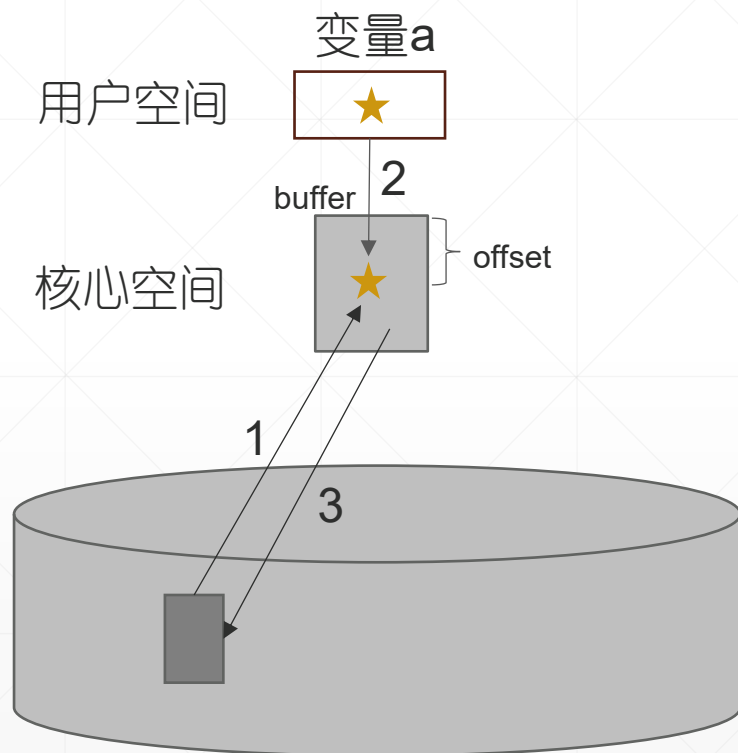


从磁盘读1个字节，赋值应用程序变量a

- 确定数据data所在的磁盘数据块号blkno，计算块内偏移量offset
- 搜索磁盘高速缓存池。blkno缓存命中？
 - Y，无需IO，直接使用buffer中的数据块复本
 - N，为blkno分配一个“空白”缓存块buffer，同步读入blkno（会睡）
- 送数据至用户区（数据传输）

`a = buffer[offset];`

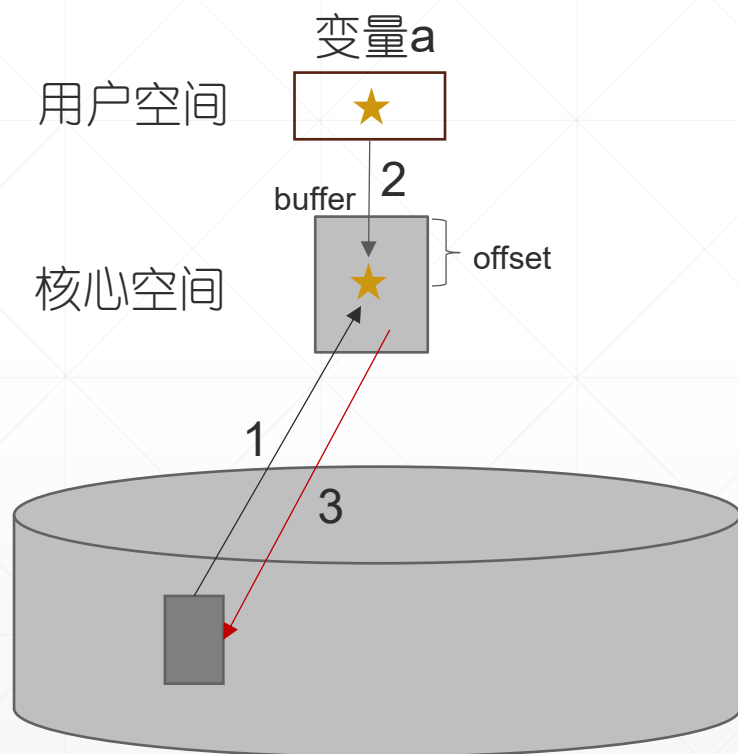
借助缓存的写操作（不考虑并发）



将应用程序变量a的值写入磁盘

- 确定需要写入的磁盘数据块号blkno，计算块内偏移量offset
- 先读
 - 1、将 blkno 同步读入缓存池，buffer缓存块
- 后写：
 - 2、a变量写入缓存池 `buffer[offset] = a;`
 - 3、整个缓存块写回磁盘

写性能优化，延迟写技术



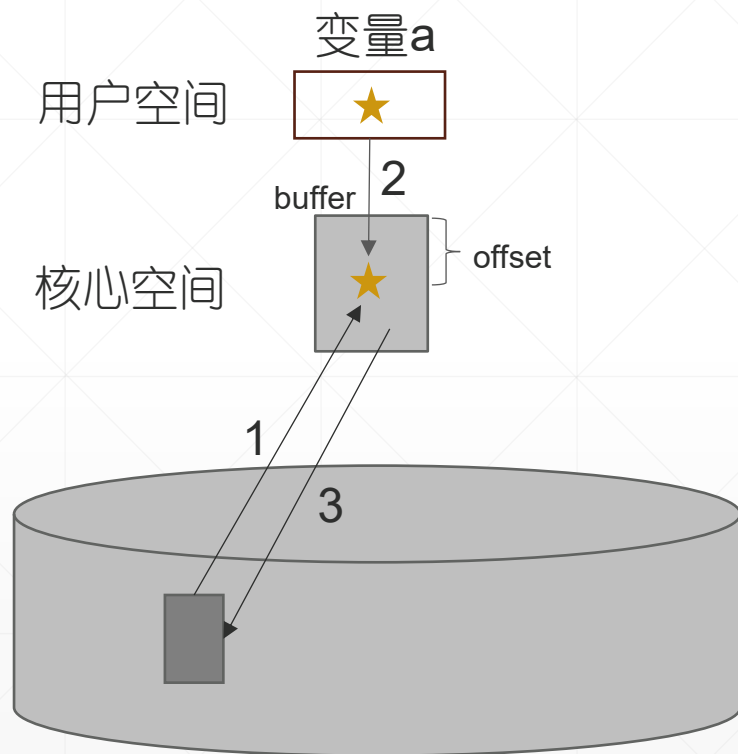
将应用程序变量a的值写入磁盘

- 确定需要写入的磁盘数据块号blkno，计算块内偏移量offset
- 先读
 - 1、将 blkno 同步读入缓存池，buffer缓存块
- 后写：
 - 2、a变量写入缓存池 $\text{buffer}[\text{offset}] = a;$

磁盘写操作（整个缓存块写回磁盘操作）可以延迟，至

buffer分配给其它磁盘数据块
磁盘卸载，电脑关机

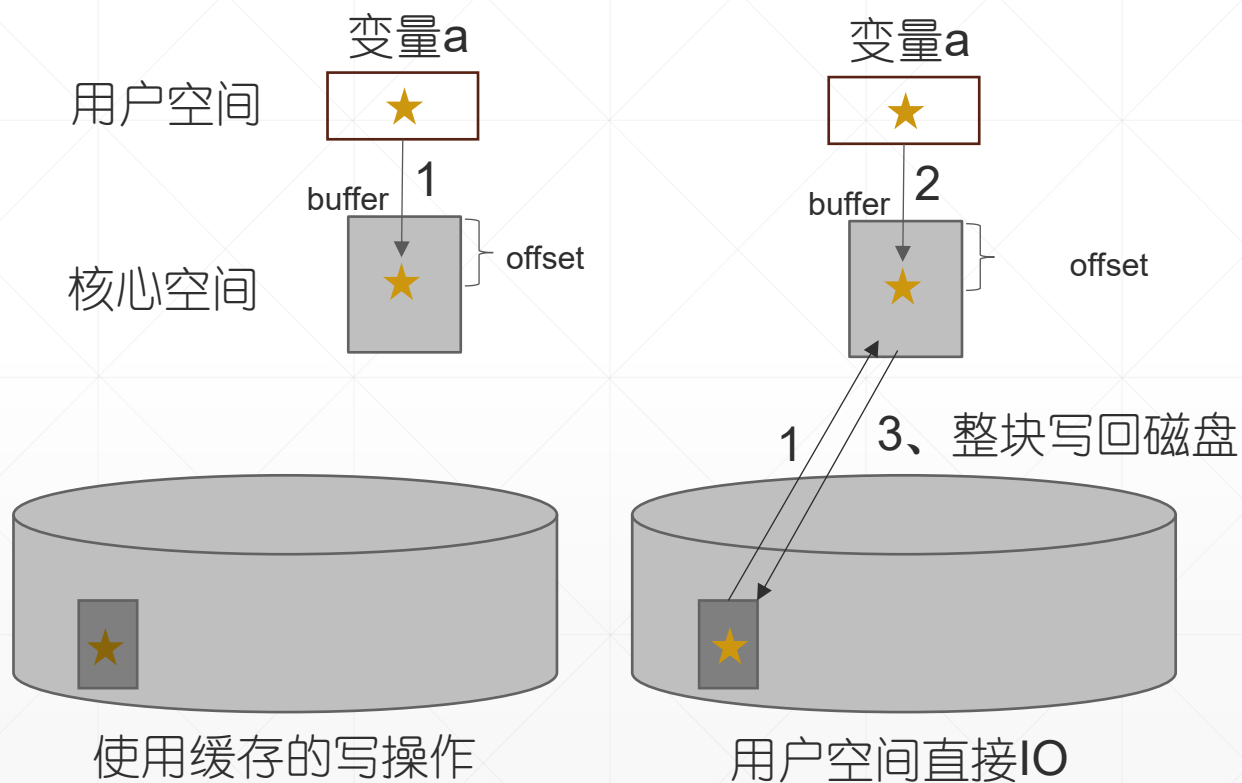
借助缓存的写操作（不考虑并发，完整版）



将尺寸为size（不越界，不大于数据块长度）数据块写入磁盘

- 确定需要写入的磁盘数据块号blkno，计算块内起始偏移量offset
- size == 数据块长度?
 - N，先读（将 blkno 同步读入缓存池，buffer缓存块）
 - Y，无需先读，为blkno分配缓存块buffer（新内容会覆盖buffer）
- 数据写入缓存池 从这里开始写：buffer[offset]
- 缓存块写满了嘛（写到最后一个字节）？
 - Y，不延迟写，立即将写请求送IO请求队列
 - N，延迟写，返回

性能 for 写操作



计算延迟写操作耗时

- 若缓存命中, t 。缓存不命中, $T + t$ 。

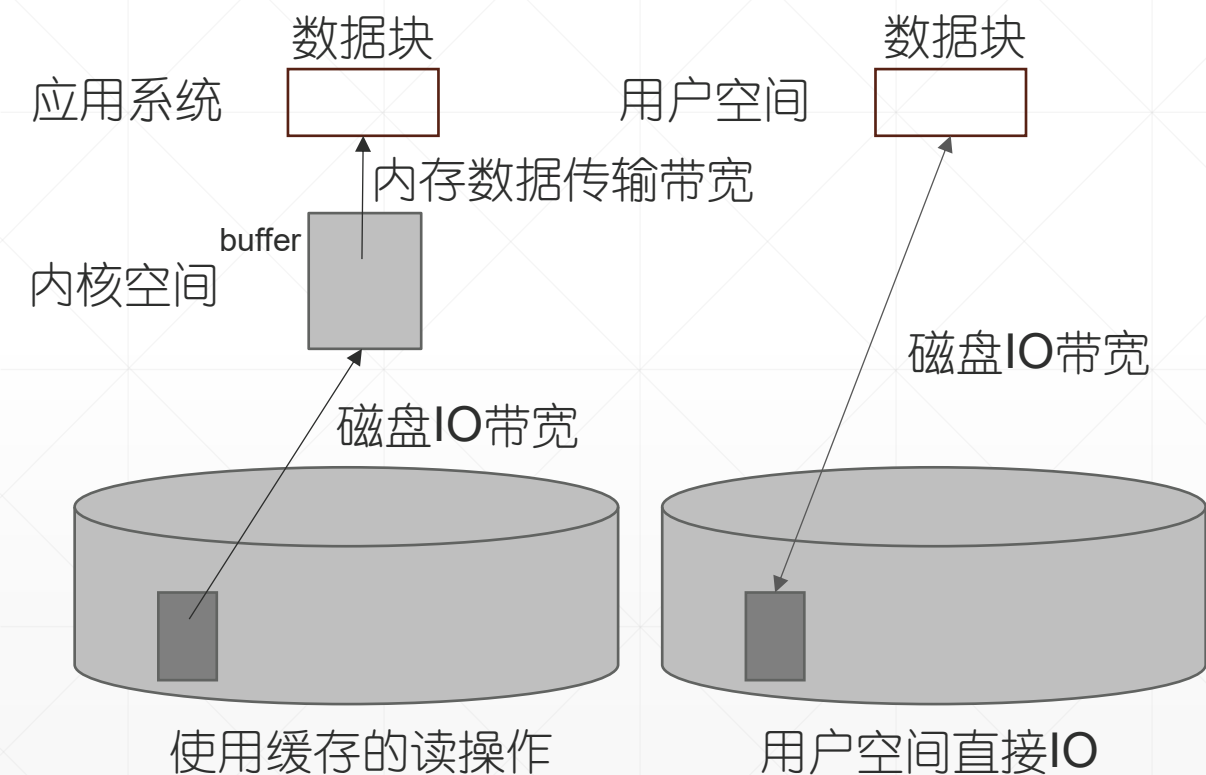
- 若缓存命中率为 95%, 写磁盘

$$\text{平均耗时} = (T + t) * 5\% + t * 95\%$$

$$= T * 5\% + t \ll 2T + t \text{ (用户空间直接IO)}$$

- 缓存的价值** (1) 缩短写操作耗时
(2) 减少写操作数量, 提高IO操作的效率
- 代价: 丢数据, 降低文件系统一致性。

用户空间IO的使用场合，其一



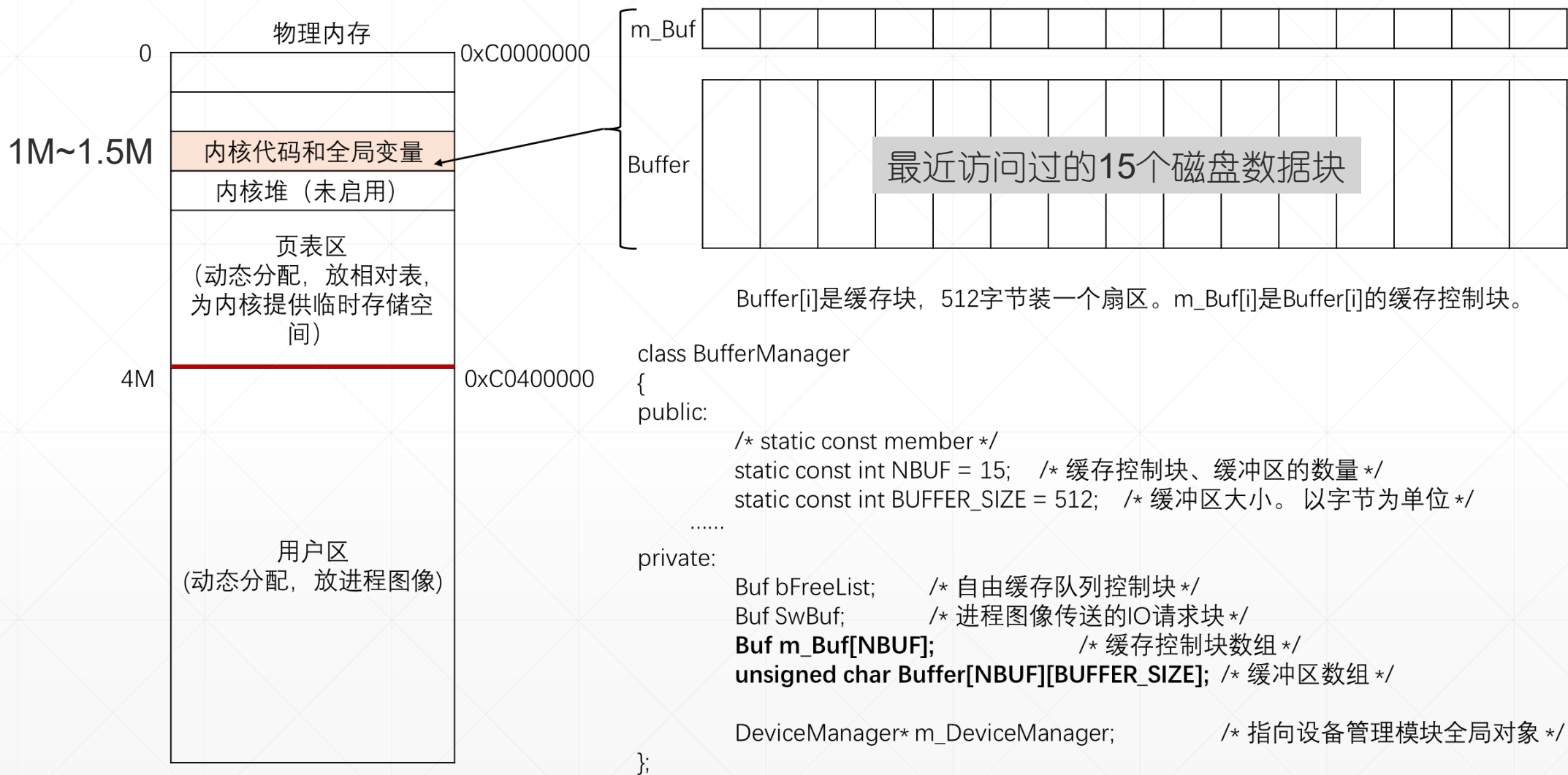
大数据批处理任务

批处理（遍历）G、T 规模的大数据集
(应用的特征：数据只用一次，没有重用可能)

关闭缓存层，消除大数据集在内存中的复制开销。

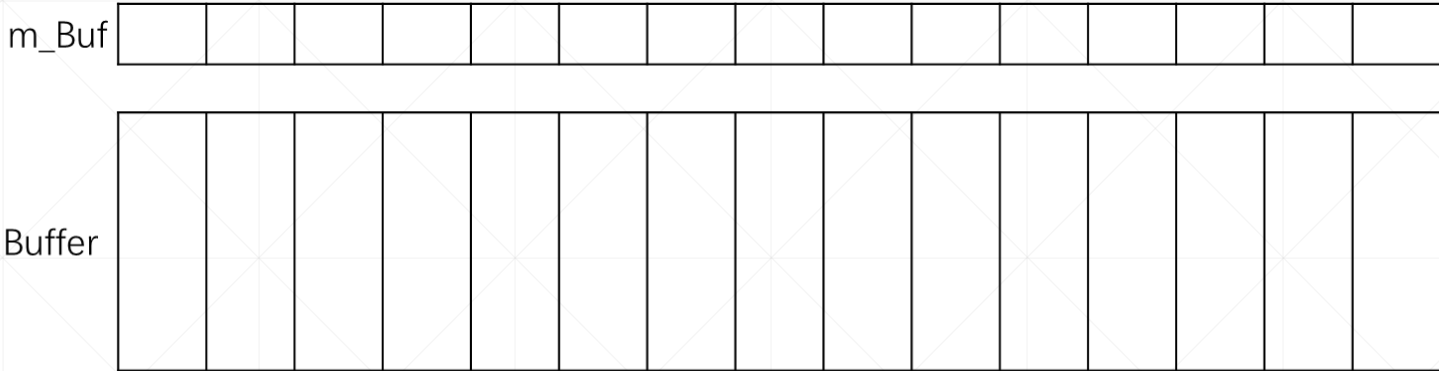
避免先读后写操作：数据文件不可更改 (immutable)
若有更新，新数据写入新数据块，甚至是新文件。

4、Unix V6++的磁盘高速缓存池





缓存控制块



Buffer[i]是缓存块，512字节装一个扇区。m_Buf[i]是Buffer[i]的缓存控制块。

```
class Buf
{ // 管理缓存块：使用状态 和 IO参数。
public:
    unsigned int b_flags; // 位图，登记缓存使用状态
    int padding; // 技术细节，看代码

    Buf* b_forw; // 队列指针，用来管理缓存
    Buf* b_back;
    Buf* av_forw;
    Buf* av_back;

    unsigned char* b_addr; // Buffer[i] 首地址
    short b_dev; // 磁盘号
    int b_blkno; // 扇区号
    int b_wcount; // IO参数，传送字节数
    int b_error; // I/O错误info
    int b_resid; // I/O出错时尚未传送的剩余字节数
};
```

```
enum BufFlag /* b_flags中标志位 */
{
    B_WRITE = 0x1,
    B_READ = 0x2, // 1#bit: 为1读操作，为0写操作
    B_DONE = 0x4, // 数据块可用，是磁盘数据块最新复本
    B_ERROR = 0x8, // I/O出错
    B_BUSY = 0x10, // 缓存正在使用，锁上了
    B_WANTED = 0x20, // 有进程等待使用数据块
    B_ASYNC = 0x40, // 异步I/O，没有进程等待IO完成
    B_DELWRI = 0x80 // 脏标识
};
```

